1. Project Overview

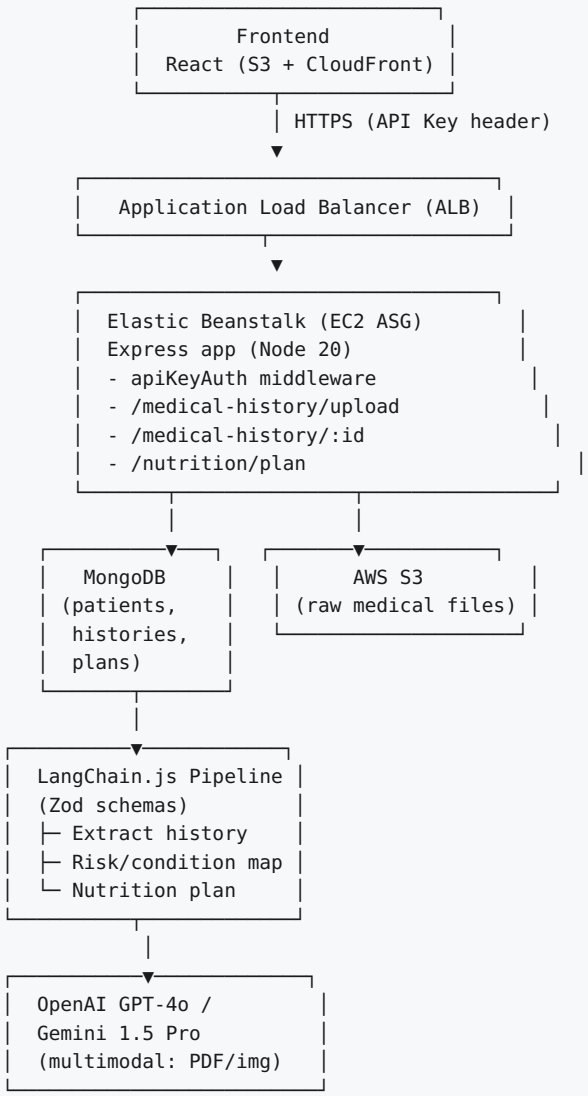
Goal: A backend service that accepts a patient's medical history (as a form, PDF, or scanned document), extracts structured medical data using a multimodal LLM, and returns a nutrition plan: foods/nutrients to avoid, foods/nutrients to favor, and general health tips tailored to the patient's conditions.

> **Important disclaimer to surface in the product itself:** this tool provides general nutrition information, not medical advice, and every response should tell the user to consult a registered dietitian or physician before making dietary changes — this is baked into the system prompt and into the UI (Section 13).

Item	Detail
Language	TypeScript (Node.js 20)
API Framework	Express
Database	MongoDB (Atlas or self-hosted), via Mongoose
LLM Orchestration	LangChain.js, output validated with **Zod** schemas
LLM Providers	OpenAI (`gpt-4o`) and Google Gemini (`gemini-1.5-pro`) — multimodal, pluggable
Auth	API Key (header-based)
Storage	AWS S3 (raw medical history files)
Tests	Jest (unit + integration), `supertest` for HTTP-level tests
CI/CD	GitHub Actions — separate `dev` and `prod` pipelines
Backend Hosting	AWS Elastic Beanstalk (behind an Application Load Balancer)
Frontend Hosting	S3 + CloudFront, deployed via CodePipeline
Secrets	AWS Secrets Manager

---

2. Architecture



3. Folder Structure

```
ai-dietitian-api/
├── src/
│   ├── index.ts                    # Express app bootstrap
│   ├── config/
│   │   └── env.ts                  # Loads env vars, fetches secrets from Secrets Manager
│   ├── middleware/
│   │   ├── apiKeyAuth.ts
│   │   └── errorHandler.ts
│   ├── models/
│   │   ├── Patient.ts              # Mongoose schema
│   │   ├── MedicalHistory.ts
│   │   └── NutritionPlan.ts
│   ├── schemas/                    # Zod schemas (LLM structured-output contracts)
│   │   ├── medicalHistorySchema.ts
│   │   └── nutritionPlanSchema.ts
│   ├── routes/
│   │   ├── medicalHistory.routes.ts
│   │   └── nutrition.routes.ts
│   ├── controllers/
│   │   ├── medicalHistory.controller.ts
│   │   └── nutrition.controller.ts
│   ├── services/
│   │   ├── s3.service.ts
│   │   ├── llm.service.ts          # LangChain.js chains
│   │   ├── mongo.service.ts
│   │   └── fileParsing.service.ts  # PDF/image normalization
│   └── types/
│       └── index.ts
├── tests/
│   ├── unit/
│   │   ├── llm.service.test.ts
│   │   ├── s3.service.test.ts
│   │   └── schemas.test.ts
│   └── integration/
│       ├── medicalHistory.upload.test.ts
│       └── nutrition.plan.test.ts
├── .ebextensions/
│   └── 01_environment.config
├── .github/
│   └── workflows/
│       ├── dev.yml
│       └── prod.yml
├── frontend/                       # Simple React UI (Section 13)
├── Dockerfile
├── jest.config.ts
├── package.json
├── tsconfig.json
└── README.md
```

---

4. Data Models (Mongoose)

```ts
// src/models/Patient.ts
import { Schema, model, Document } from "mongoose";

export interface IPatient extends Document {
  fullName: string;
  age?: number;
  sex?: "male" | "female" | "other";
  medicalHistoryS3Key: string;
  originalFilename: string;
  uploadedAt: Date;
  status: "processing" | "analyzed" | "failed";
}

const PatientSchema = new Schema<IPatient>({
  fullName: { type: String, required: true },
  age: Number,
  sex: { type: String, enum: ["male", "female", "other"] },
  medicalHistoryS3Key: { type: String, required: true },
  originalFilename: { type: String, required: true },
  uploadedAt: { type: Date, default: Date.now },
  status: { type: String, enum: ["processing", "analyzed", "failed"], default: "processing" },
});

export default model<IPatient>("Patient", PatientSchema);
```

```ts
// src/models/NutritionPlan.ts
import { Schema, model, Document } from "mongoose";

export interface INutritionPlan extends Document {
  patientId: Schema.Types.ObjectId;
  extractedHistory: Record<string, unknown>;
  foodsToAvoid: Record<string, unknown>[];
  foodsToEat: Record<string, unknown>[];
  healthTips: string[];
  createdAt: Date;
}

const NutritionPlanSchema = new Schema<INutritionPlan>({
  patientId: { type: Schema.Types.ObjectId, ref: "Patient", required: true },
  extractedHistory: { type: Schema.Types.Mixed, required: true },
  foodsToAvoid: [{ type: Schema.Types.Mixed }],
  foodsToEat: [{ type: Schema.Types.Mixed }],
  healthTips: [String],
  createdAt: { type: Date, default: Date.now },
});

export default model<INutritionPlan>("NutritionPlan", NutritionPlanSchema);
```

---

5. LLM Output Schemas (Zod)

```ts
// src/schemas/medicalHistorySchema.ts
import { z } from "zod";

export const ConditionSchema = z.object({
  name: z.string().describe("e.g. Type 2 Diabetes, Hypertension"),
  status: z.enum(["active", "managed", "resolved"]),
  diagnosedYear: z.string().optional(),
});

export const MedicationSchema = z.object({
  name: z.string(),
  dosage: z.string().optional(),
  purpose: z.string().optional(),
});

export const MedicalHistorySchema = z.object({
  fullName: z.string(),
  age: z.number().optional(),
  sex: z.enum(["male", "female", "other"]).optional(),
  conditions: z.array(ConditionSchema),
  medications: z.array(MedicationSchema),
  allergies: z.array(z.string()).default([]),
  labResults: z
    .array(
      z.object({
        test: z.string(),
        value: z.string(),
        unit: z.string().optional(),
        flag: z.enum(["low", "normal", "high"]).optional(),
      })
    )
    .default([]),
  familyHistory: z.array(z.string()).default([]),
  lifestyle: z
    .object({
      smoker: z.boolean().optional(),
      alcoholUse: z.string().optional(),
      activityLevel: z.string().optional(),
    })
    .optional(),
});

export type MedicalHistory = z.infer<typeof MedicalHistorySchema>;
```

```
        // src/schemas/nutritionPlanSchema.ts
        import { z } from "zod";

        export const FoodItemSchema = z.object({
          item: z.string(),
          reason: z.string(),
          relatedCondition: z.string().optional(),
        });

        export const NutritionPlanSchema = z.object({
          summary: z.string().describe("2-3 sentence overview of the patient's nutritional priorities"),
          foodsToAvoid: z.array(FoodItemSchema),
          nutrientsToLimit: z.array(FoodItemSchema),
          foodsToEat: z.array(FoodItemSchema),
          nutrientsToIncrease: z.array(FoodItemSchema),
          healthTips: z.array(z.string()),
          disclaimer: z
            .string()
            .default(
              "This is general nutrition information generated by AI, not medical advice. Consult a registered
dietitian or your physician before changing your diet."
            ),
        });

        export type NutritionPlan = z.infer<typeof NutritionPlanSchema>;
```

---

6. LangChain.js Service (Multimodal Extraction + Structured Output)

```ts
// src/services/llm.service.ts
import { ChatOpenAI } from "@langchain/openai";
import { ChatGoogleGenerativeAI } from "@langchain/google-genai";
import { HumanMessage } from "@langchain/core/messages";
import { MedicalHistorySchema, MedicalHistory } from "../schemas/medicalHistorySchema";
import { NutritionPlanSchema, NutritionPlan } from "../schemas/nutritionPlanSchema";
import { env } from "../config/env";

type Provider = "openai" | "gemini";

function getLLM(provider: Provider = "openai") {
  if (provider === "gemini") {
    return new ChatGoogleGenerativeAI({ model: "gemini-1.5-pro", apiKey: env.GEMINI_API_KEY });
  }
  return new ChatOpenAI({ model: "gpt-4o", apiKey: env.OPENAI_API_KEY, temperature: 0 });
}

export async function extractMedicalHistory(
  fileBase64: string,
  mimeType: string,
  provider: Provider = "openai"
): Promise<MedicalHistory> {
  const llm = getLLM(provider).withStructuredOutput(MedicalHistorySchema);
  const message = new HumanMessage({
    content: [
      { type: "text", text: "Extract all medical history details from this document into the structured schema." },
      { type: "image_url", image_url: `data:${mimeType};base64,${fileBase64}` },
    ],
  });
  return llm.invoke([message]);
}

export async function generateNutritionPlan(
  history: MedicalHistory,
  provider: Provider = "openai"
): Promise<NutritionPlan> {
  const llm = getLLM(provider).withStructuredOutput(NutritionPlanSchema);
  const systemPrompt = `You are a registered-dietitian assistant. Given a patient's structured medical history (conditions, medications, allergies, lab results), produce a practical nutrition plan: specific foods/nutrients to avoid or limit (and why, tied to their condition), foods/nutrients to prioritize, and general health tips. Be conservative and evidence-based (e.g. low-sodium for hypertension, low-glycemic-index carbs for diabetes, avoid grapefruit with certain statins/interactions). Always include the disclaimer field.`;

  const message = new HumanMessage(
    `${systemPrompt}\n\nPatient history:\n${JSON.stringify(history)}`
  );
  return llm.invoke([message]);
}
```

7. API Endpoints

Method	Path	Description	Auth
POST	`/medical-history/upload`	Multipart upload of medical history file (or JSON form body) → stores raw file in S3, creates `Patient` doc, triggers async analysis	API Key
GET	`/medical-history/:id`	Get patient metadata + status	API Key
GET	`/nutrition/:patientId/plan`	Get generated nutrition plan	API Key

POST	`/nutrition/plan`	Body: `{ patientId }` → (re)generate plan on demand	API Key
GET	`/health`	Liveness/readiness probe for ALB	none

Sample Route + Controller

```
// src/routes/medicalHistory.routes.ts
import { Router } from "express";
import multer from "multer";
import { apiKeyAuth } from "../middleware/apiKeyAuth";
import { uploadMedicalHistory, getPatient } from "../controllers/medicalHistory.controller";

const upload = multer({ storage: multer.memoryStorage(), limits: { fileSize: 10 * 1024 * 1024 } });
const router = Router();

router.post("/upload", apiKeyAuth, upload.single("file"), uploadMedicalHistory);
router.get("/:id", apiKeyAuth, getPatient);

export default router;
```

```
// src/controllers/medicalHistory.controller.ts
import { Request, Response, NextFunction } from "express";
import Patient from "../models/Patient";
import { uploadFile } from "../services/s3.service";
import { extractMedicalHistory, generateNutritionPlan } from "../services/llm.service";
import NutritionPlan from "../models/NutritionPlan";
import { toBase64Image } from "../services/fileParsing.service";

export async function uploadMedicalHistory(req: Request, res: Response, next: NextFunction) {
  try {
    if (!req.file) return res.status(400).json({ error: "file is required" });

    const s3Key = await uploadFile(req.file.buffer, req.file.originalname);
    const patient = await Patient.create({
      fullName: req.body.fullName ?? "Unknown",
      medicalHistoryS3Key: s3Key,
      originalFilename: req.file.originalname,
    });

    res.status(202).json({ patientId: patient._id, status: "processing" });

    // Fire-and-forget background pipeline (in production: move to SQS + worker)
    processHistoryAsync(patient._id.toString(), req.file.buffer, req.file.mimetype).catch(console.error);
  } catch (err) {
    next(err);
  }
}

async function processHistoryAsync(patientId: string, buffer: Buffer, mimeType: string) {
  const { base64, normalizedMime } = await toBase64Image(buffer, mimeType);
  const history = await extractMedicalHistory(base64, normalizedMime);
  const plan = await generateNutritionPlan(history);

  await NutritionPlan.create({ patientId, extractedHistory: history, ...plan });
  await Patient.findByIdAndUpdate(patientId, { status: "analyzed" });
}

export async function getPatient(req: Request, res: Response, next: NextFunction) {
  try {
    const patient = await Patient.findById(req.params.id);
    if (!patient) return res.status(404).json({ error: "Not found" });
    res.json(patient);
  } catch (err) {
    next(err);
  }
}
```

8. API Key Authentication Middleware

```
// src/middleware/apiKeyAuth.ts
import { Request, Response, NextFunction } from "express";
import crypto from "crypto";
import { env } from "../config/env";

export function apiKeyAuth(req: Request, res: Response, next: NextFunction) {
  const provided = req.header("x-api-key") ?? "";
  const expected = env.API_KEY;

  const a = Buffer.from(provided);
  const b = Buffer.from(expected);
  const valid = a.length === b.length && crypto.timingSafeEqual(a, b);

  if (!valid) {
    return res.status(401).json({ error: "Invalid API key" });
  }
  next();
}
```

9. S3 Upload Service

```
// src/services/s3.service.ts
import { S3Client, PutObjectCommand, GetObjectCommand } from "@aws-sdk/client-s3";
import { getSignedUrl } from "@aws-sdk/s3-request-presigner";
import { randomUUID } from "crypto";
import { env } from "../config/env";

const s3 = new S3Client({ region: env.AWS_REGION });

export async function uploadFile(buffer: Buffer, originalFilename: string): Promise<string> {
  const key = `medical-history/${randomUUID()}-${originalFilename}`;
  await s3.send(new PutObjectCommand({ Bucket: env.S3_BUCKET, Key: key, Body: buffer }));
  return key;
}

export async function getPresignedUrl(key: string, expiresIn = 3600): Promise<string> {
  const command = new GetObjectCommand({ Bucket: env.S3_BUCKET, Key: key });
  return getSignedUrl(s3, command, { expiresIn });
}
```

10. Testing Strategy (Jest)

- **Unit tests**: mock `@aws-sdk/client-s3`, the LangChain chat model classes, and Mongoose models (via `mongodb-memory-server` or manual mocks) to test services in isolation.
- **Integration tests**: use `supertest` against the Express app with an in-memory MongoDB (`mongodb-memory-server`), LLM calls mocked with `jest.mock`.
- Coverage target: **≥ 80%**, enforced via `jest --coverage --coverageThreshold`.

```ts
// tests/unit/schemas.test.ts
import { MedicalHistorySchema } from "../../src/schemas/medicalHistorySchema";

describe("MedicalHistorySchema", () => {
  it("parses a valid medical history payload", () => {
    const result = MedicalHistorySchema.safeParse({
      fullName: "Kofi Mensah",
      age: 54,
      sex: "male",
      conditions: [{ name: "Type 2 Diabetes", status: "active" }],
      medications: [{ name: "Metformin", dosage: "500mg" }],
      allergies: [],
    });
    expect(result.success).toBe(true);
  });

  it("rejects a payload missing required fields", () => {
    const result = MedicalHistorySchema.safeParse({ age: 54 });
    expect(result.success).toBe(false);
  });
});
```

```ts
// tests/integration/medicalHistory.upload.test.ts
import request from "supertest";
import app from "../../src/index";
import * as s3Service from "../../src/services/s3.service";

jest.mock("../../src/services/s3.service");
jest.mock("../../src/services/llm.service");

describe("POST /medical-history/upload", () => {
  it("accepts a file upload and returns processing status", async () => {
    (s3Service.uploadFile as jest.Mock).mockResolvedValue("medical-history/fake-key.pdf");

    const response = await request(app)
      .post("/medical-history/upload")
      .set("x-api-key", process.env.API_KEY as string)
      .field("fullName", "Kofi Mensah")
      .attach("file", Buffer.from("fake pdf content"), "history.pdf");

    expect(response.status).toBe(202);
    expect(response.body.status).toBe("processing");
  });

  it("rejects requests without a valid API key", async () => {
    const response = await request(app).post("/medical-history/upload").attach("file", Buffer.from("x"),
"a.pdf");
    expect(response.status).toBe(401);
  });
});
```

```ts
// jest.config.ts
import type { Config } from "jest";

const config: Config = {
  preset: "ts-jest",
  testEnvironment: "node",
  coveragePathIgnorePatterns: ["/node_modules/"],
  coverageThreshold: {
    global: { branches: 75, functions: 80, lines: 80, statements: 80 },
  },
};
export default config;
```

11. AWS Infrastructure

11.1 S3 (raw medical history storage)

- Bucket: `ai-dietitian-uploads-{env}` — versioning **on**, default encryption **SSE-KMS** (medical data is sensitive — prefer KMS over SSE-S3 for auditability), block all public access.
- Access logging enabled to a separate log bucket for compliance/audit trail.

11.2 Secrets Manager

- Secret name: `ai-dietitian/{env}/app-secrets` — JSON blob with `OPENAI_API_KEY`, `GEMINI_API_KEY`, `MONGODB_URI`, `API_KEY`.
- EC2 instance role scoped via IAM policy to `secretsmanager:GetSecretValue` on this ARN only.

11.3 Elastic Beanstalk (backend)

- Platform: **Docker on Amazon Linux 2023** (Node 20 base image).
- Environment tier: Web server, load-balanced, min 2 / max 4 instances, scale on CPU > 60% or request count per target.

```
# .ebextensions/01_environment.config
container_commands:
  01_fetch_secrets:
    command: "node scripts/load-secrets.js"
option_settings:
  aws:autoscaling:asg:
    MinSize: 2
    MaxSize: 4
  aws:elasticbeanstalk:environment:
    LoadBalancerType: application
  aws:elasticbeanstalk:healthreporting:system:
    SystemType: enhanced
```

11.4 Application Load Balancer

- Listener 443 (ACM cert) → target group on container port 3000.
- Health check path: `/health`.

11.5 EC2

- Instance type: `t3.small` (dev) / `t3.medium` (prod), private subnet, outbound via NAT Gateway for LLM API calls.

11.6 CodePipeline (backend deploy)

1. **Source**: GitHub (CodeStar connection), triggers on push to `main` (prod) / `develop` (dev).
2. **Build**: CodeBuild — `npm ci`, `npm run build`, `npm test`, `docker build`, push to ECR.
3. **Deploy**: Elastic Beanstalk deploy action per environment.

---

12. GitHub Actions (Dev / Prod)

```yaml
# .github/workflows/dev.yml
name: Deploy Dev
on:
  push:
    branches: [develop]
jobs:
  test-and-deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: "20" }
      - run: npm ci
      - run: npm run build
      - run: npm test -- --coverage
      - name: Configure AWS credentials
        uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: ${{ secrets.AWS_ROLE_DEV }}
          aws-region: eu-west-1
      - name: Deploy to Elastic Beanstalk (dev)
        uses: einaregilsson/beanstalk-deploy@v22
        with:
          aws_access_key: ${{ secrets.AWS_ACCESS_KEY_ID }}
          aws_secret_key: ${{ secrets.AWS_SECRET_ACCESS_KEY }}
          application_name: ai-dietitian
          environment_name: ai-dietitian-dev
          region: eu-west-1
          version_label: dev-${{ github.sha }}
          deployment_package: deploy.zip
```

```yaml
# .github/workflows/prod.yml
name: Deploy Prod
on:
  push:
    branches: [main]
jobs:
  test-and-deploy:
    runs-on: ubuntu-latest
    environment: production
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: "20" }
      - run: npm ci
      - run: npm run build
      - run: npm test -- --coverage
      - name: Configure AWS credentials
        uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: ${{ secrets.AWS_ROLE_PROD }}
          aws-region: eu-west-1
      - name: Deploy to Elastic Beanstalk (prod)
        uses: einaregilsson/beanstalk-deploy@v22
        with:
          aws_access_key: ${{ secrets.AWS_ACCESS_KEY_ID }}
          aws_secret_key: ${{ secrets.AWS_SECRET_ACCESS_KEY }}
          application_name: ai-dietitian
          environment_name: ai-dietitian-prod
          region: eu-west-1
          version_label: prod-${{ github.sha }}
          deployment_package: deploy.zip
```

Both pipelines block deploy on a failing `npm test` (Jest) step, same gating pattern as the Python project for consistency across both assessment deliverables.

---

13. Frontend — Simple UI

A minimal React + TypeScript single-page app:

- **Intake screen**: a short structured form (age, sex, known conditions, medications, allergies) *plus* an optional file upload for existing medical records/lab reports — either path (or both) feeds the LLM extraction step.
- **Plan screen**: shows the generated `NutritionPlan` — a two-column "Avoid / Limit" vs "Eat / Increase" layout, each item with its condition-linked reason, a health-tips list, and the disclaimer banner rendered prominently (not just in fine print) given this is health-adjacent content.
- API key stored in `.env` (`VITE_API_KEY`), sent as `x-api-key` header.

Deployment: 1. `npm run build` → static files in `dist/` . 2. **S3** bucket `ai-dietitian-frontend-{env}` (no public website hosting; CloudFront Origin Access Control only). 3. **CloudFront** distribution, custom domain via ACM + Route 53. 4. **CodePipeline**: Source (GitHub) → Build (CodeBuild: `npm ci && npm run build`) → Deploy (S3 sync + CloudFront cache invalidation).

---

14. Environment Variables

Variable	Purpose
`MONGODB_URI`	Mongo connection string
`OPENAI_API_KEY`	OpenAI access
`GEMINI_API_KEY`	Gemini access
`API_KEY`	Shared API key for clients
`S3_BUCKET`	Raw medical-history storage bucket
`AWS_REGION`	AWS region
`PORT`	Express listen port (default 3000)

---

15. package.json (key dependencies)

```
{
  "name": "ai-dietitian-api",
  "version": "1.0.0",
  "scripts": {
    "dev": "ts-node-dev src/index.ts",
    "build": "tsc -p tsconfig.json",
    "start": "node dist/index.js",
    "test": "jest"
  },
  "dependencies": {
    "express": "^4.19.2",
    "mongoose": "^8.5.0",
    "multer": "^1.4.5-lts.1",
    "zod": "^3.23.8",
    "@langchain/core": "^0.3.0",
    "@langchain/openai": "^0.3.0",
    "@langchain/google-genai": "^0.1.0",
    "@aws-sdk/client-s3": "^3.620.0",
    "@aws-sdk/client-secrets-manager": "^3.620.0",
    "@aws-sdk/s3-request-presigner": "^3.620.0",
    "dotenv": "^16.4.5"
  },
  "devDependencies": {
    "typescript": "^5.5.4",
    "ts-node-dev": "^2.0.0",
    "ts-jest": "^29.2.4",
    "jest": "^29.7.0",
    "supertest": "^7.0.0",
    "mongodb-memory-server": "^10.0.0",
    "@types/express": "^4.17.21",
    "@types/multer": "^1.4.11",
    "@types/jest": "^29.5.12",
    "@types/supertest": "^6.0.2"
  }
}
```

16. Assessment Requirement Checklist

Requirement	Where addressed
TypeScript	Entire backend (Section 3–9)
Unit/Integration tests (Jest)	Section 10
Express	Section 3, 7
API Key Authentication	Section 8
MongoDB	Section 4, 9
GitHub Actions (Dev/Prod)	Section 12
Multimodal LLMs (Gemini/OpenAI)	Section 6
LangChain + Zod schema	Section 5, 6
AWS: CodePipeline, Elastic Beanstalk, S3, EC2, Load Balancing, Secrets Manager	Section 11
Frontend: CloudFront/S3, CodePipeline	Section 13
Simple UI	Section 13
Take medical history	Section 5 (`MedicalHistorySchema`), Section 6
Foods/nutrients to avoid & eat, health tips	Section 5 (`NutritionPlanSchema`), Section 6